

AXI (SPI & I2C) Soc 설계 및 검증

발표자 이준형



목차

01 개요

02 Soc 설계

03 AXI
모듈 설계

04 UVM 검증

05 Trouble
Shooting

06 느낀 점



개요

목표 :

1. Micro Blaze를 활용한 SPI, I2C 통신 설계
2. AXI4 Lite를 이해하고 SPI, I2C Master 송수신 제어하며 설계
3. AXI4 Lite 모듈을 UVM 검증
4. 두 보드(Basys3)에 FPGA합성하여 송수신 구현

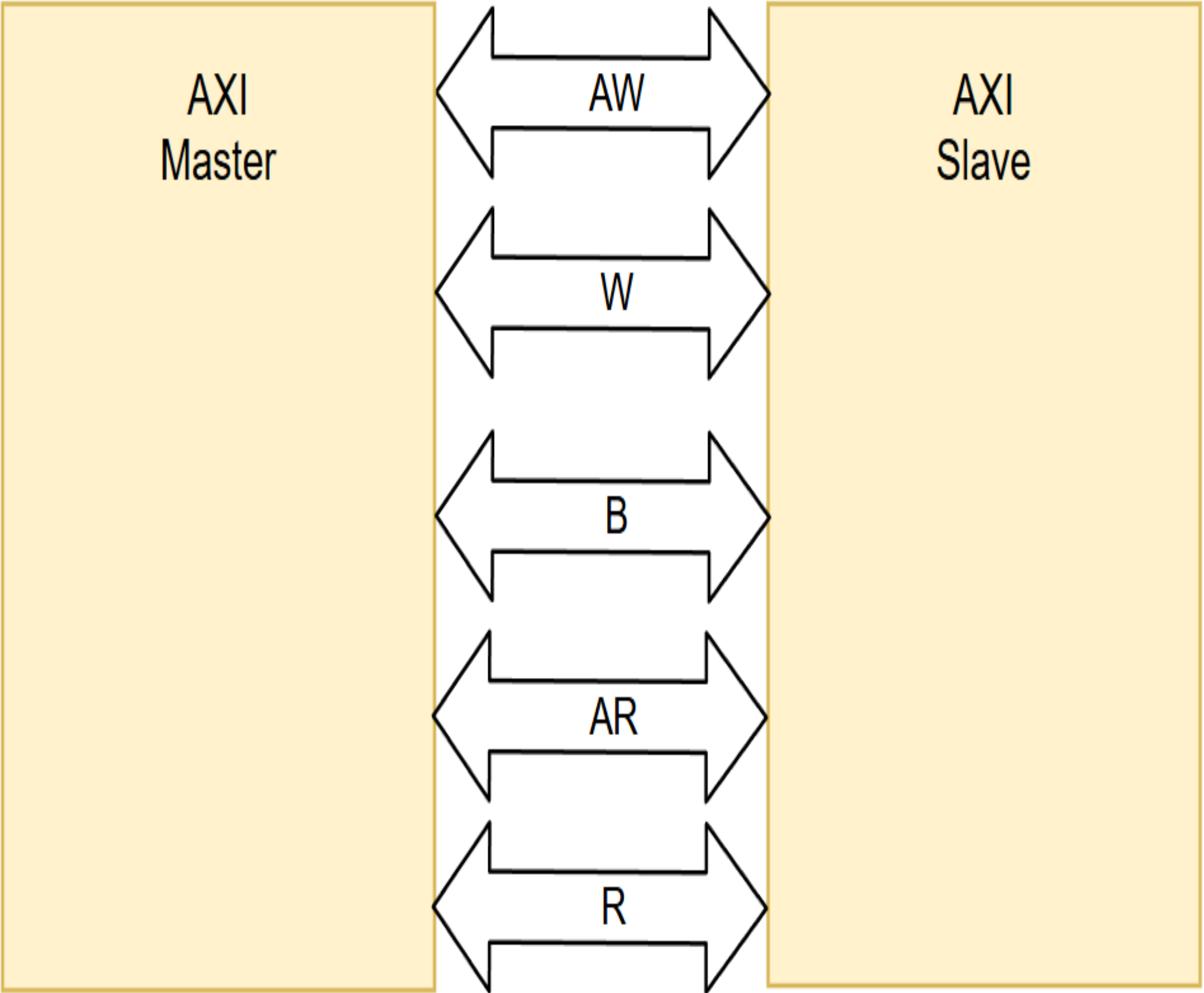
- 사용한 툴 및 환경

Soc 설계 및 FPGA 합성	VIVADO, Vigit_IDE
UVM 검증	VCS, Verdi
Board	Basys3
AXI 버전	AXI4 Lite

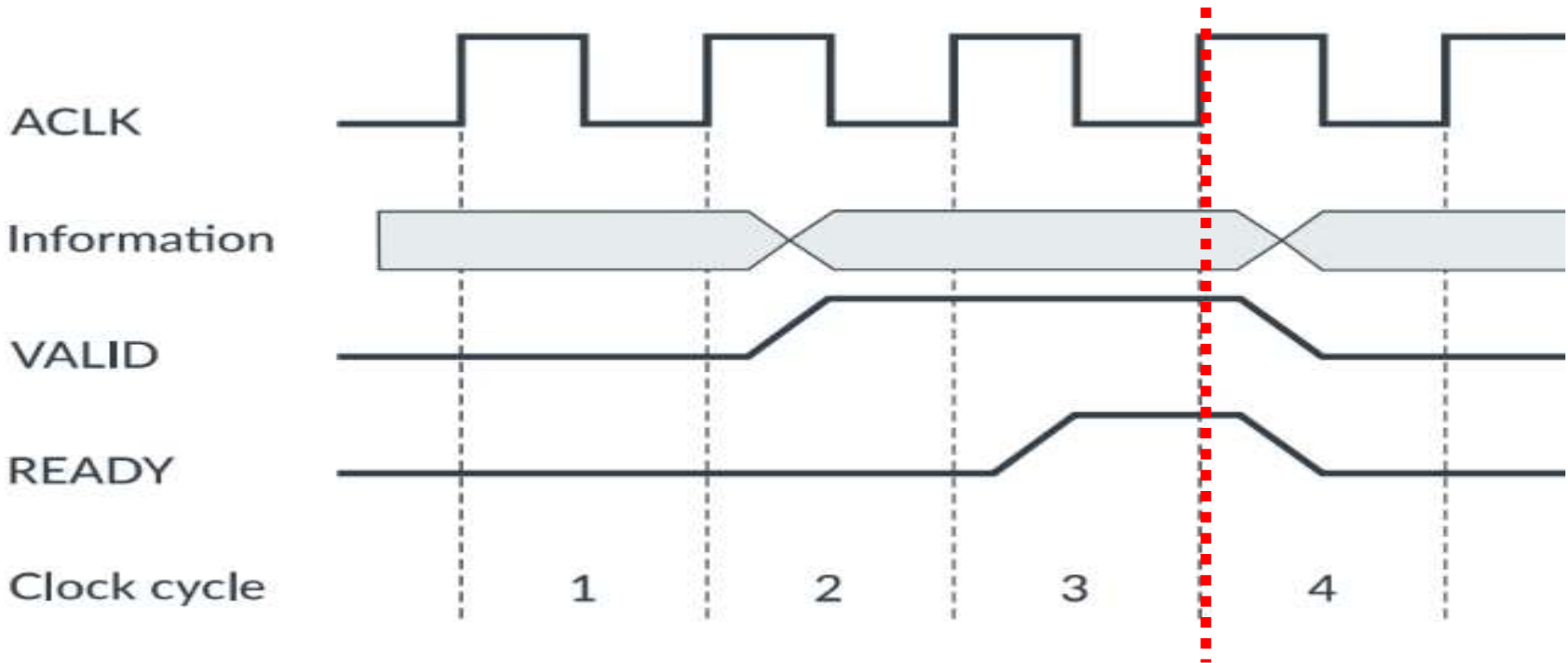
AXI

AXI :

ARM의 AMBA 사양 중 가장 널리 사용되는 고성능, 고대역폭 통신 프로토콜
5개의 독립적인 채널을 통해 양방향 통신을 수행. 각 채널은 **Ready/Valid 핸드 셰이크 동작을 수행함**

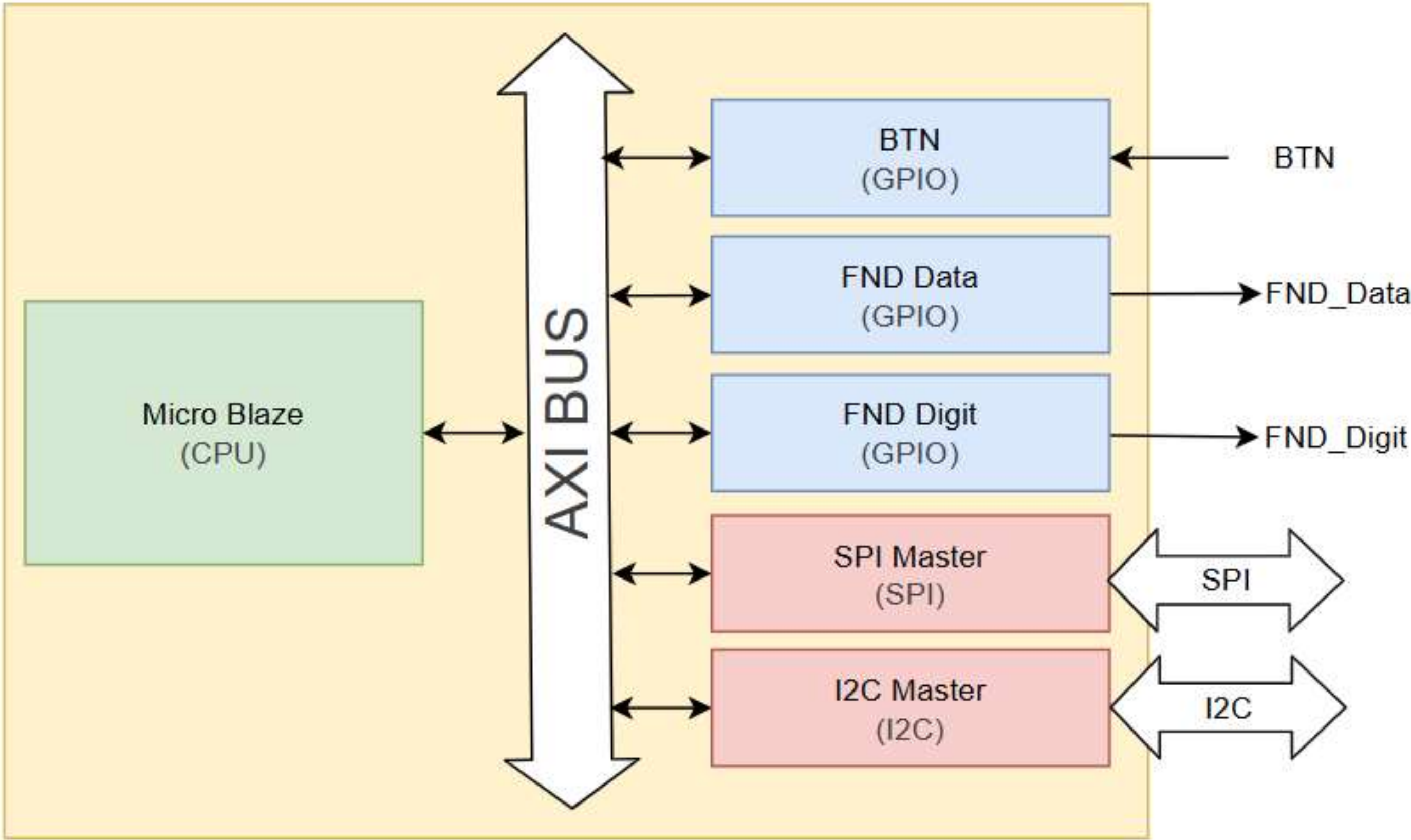


채널	정의
AW	Write 주소 전달
W	Write Data값을 보냄
B	Slave가 쓰기 완료 상태 최종 보고함
AR	Read 주소 전달
R	Read Data값을 보냄



Soc 설계

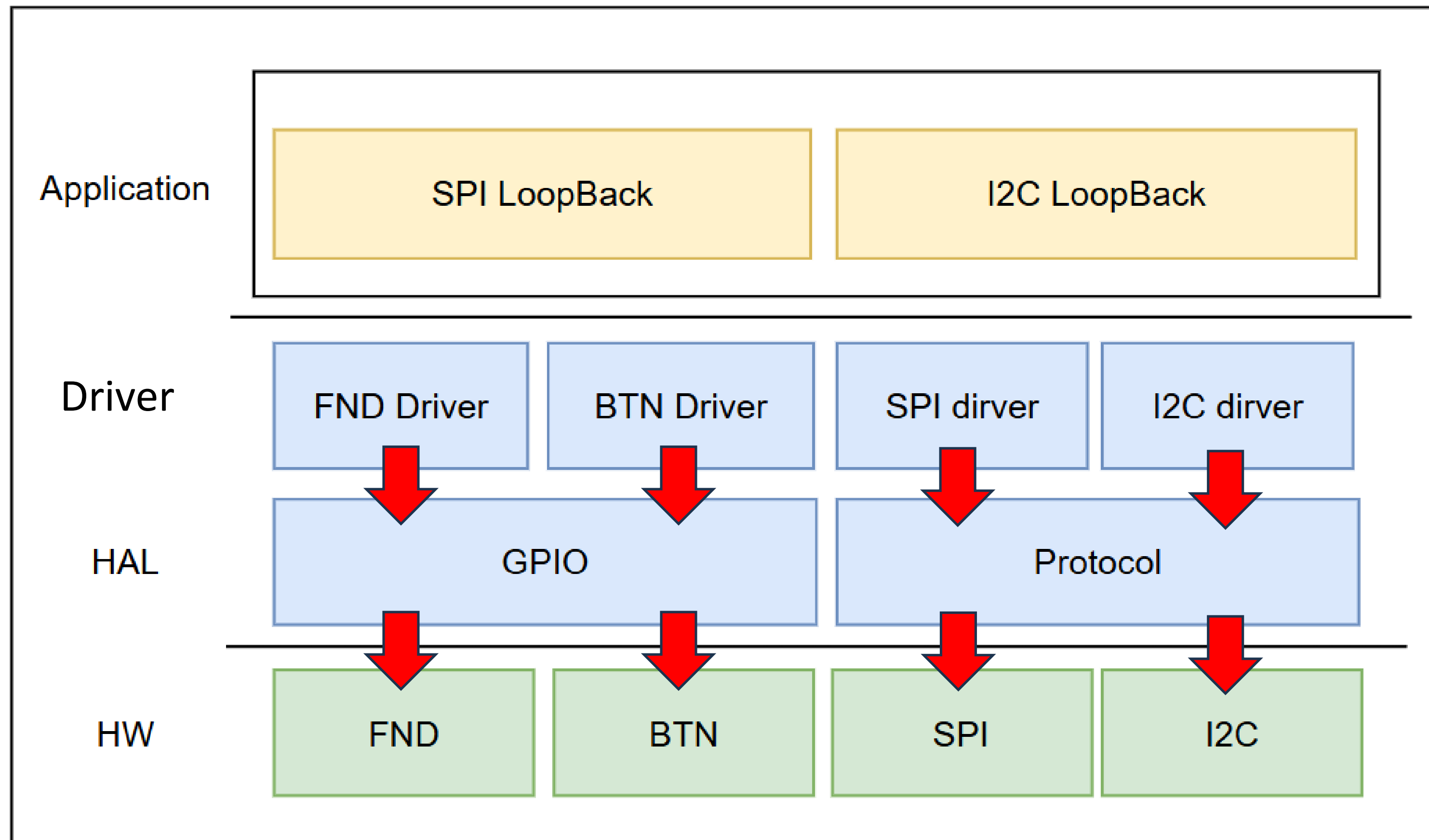
- B/D(HW)



Memory Mapping	
I2C Master	0x44A4_0000
SPI Master	0x44A3_0000
FND Digit (GPIO)	0x44A2_0000
FND Data (GPIO)	0x44A1_0000
BTN (GPIO)	0x44A0_0000
⋮	
Memory	0x0001_FFFF 0x0000_0000

Soc 설계

- Software 계층 구조



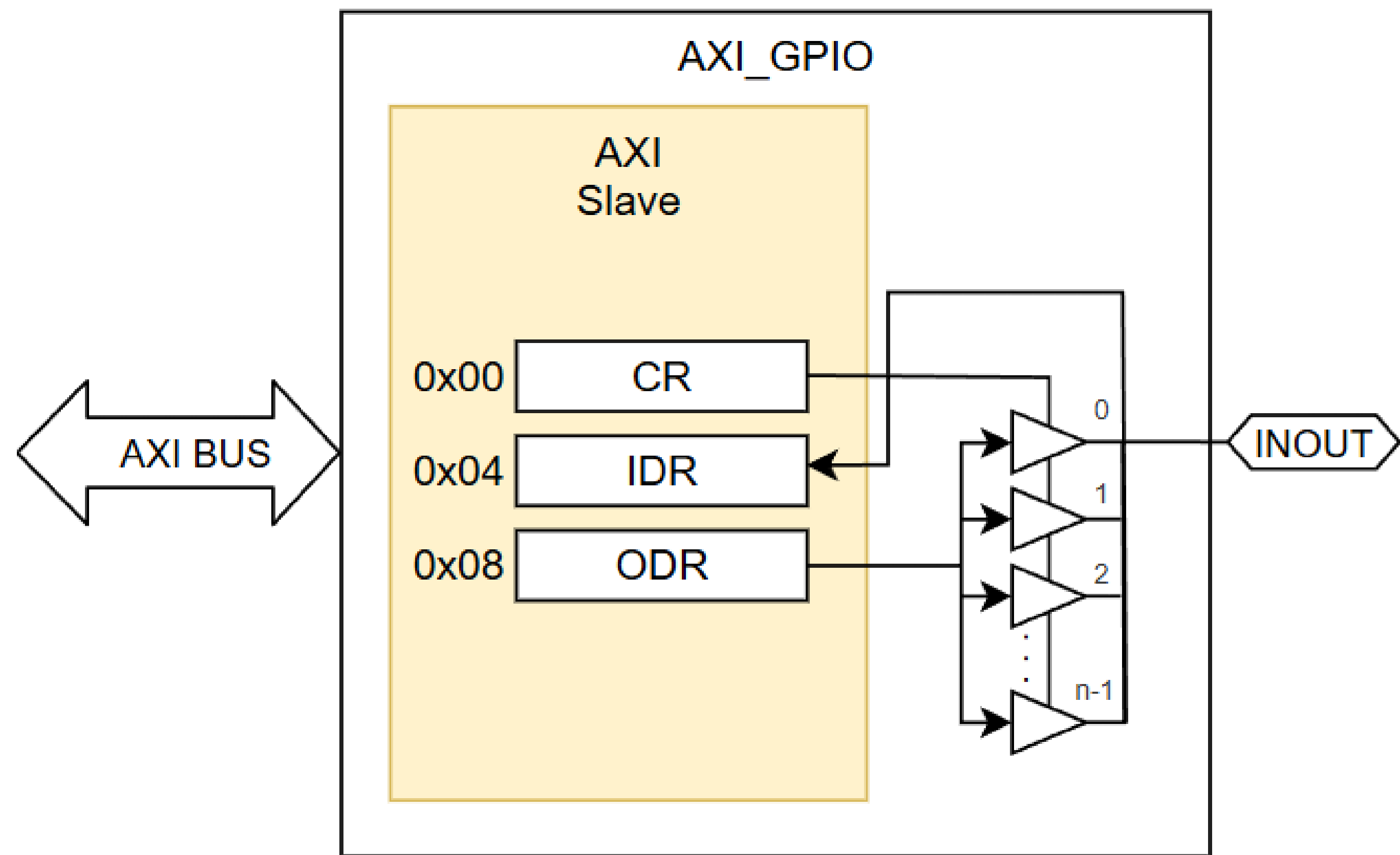
GPIO

01 모듈 설계



GPIO

- 모듈 설계



REG	Off-Set	Purpose
CR	0x00	Tri-state 버퍼 제어(bit 단위)
IDR	0x04	INPUT Data
ODR	0x08	OUTPUT Data

1. 사용자 입력 인터페이스 구현:

BTN 입력 신호를 스캔하여
시스템 제어 이벤트 생성

2. 시각적 정보 출력:

SPI, I2C 통신에서 RX_Data 표기

SPI

01 모듈 설계

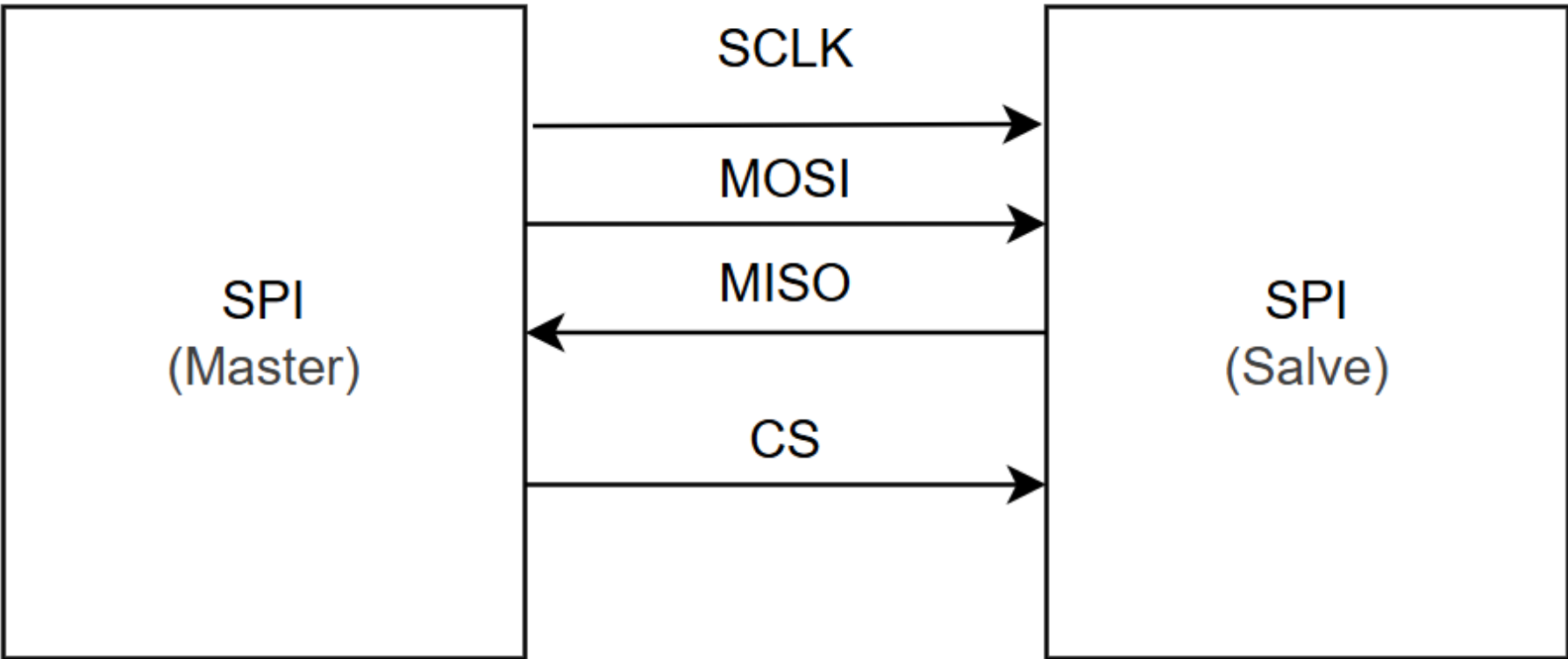
02 Application 설계

03 FPGA 구현

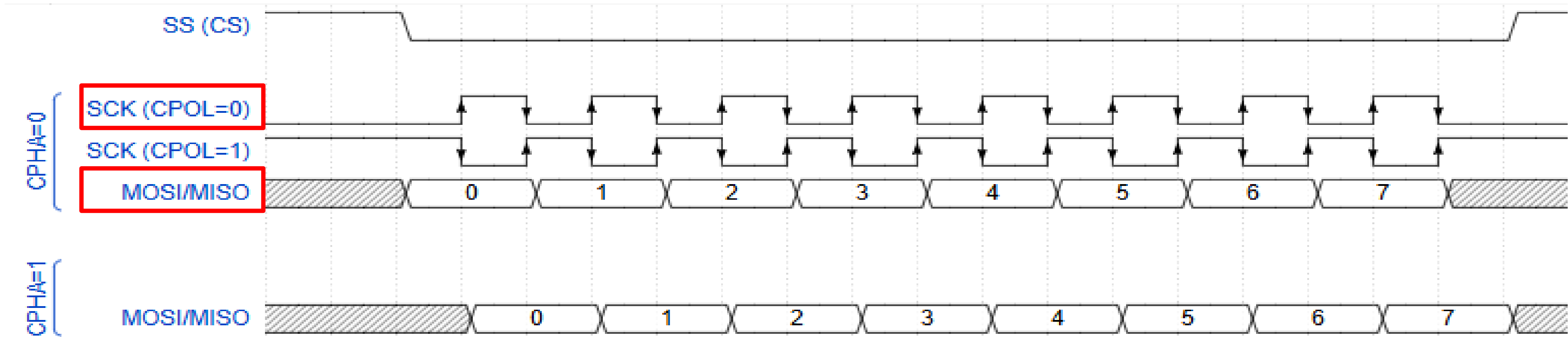


SPI

- SPI Protocol

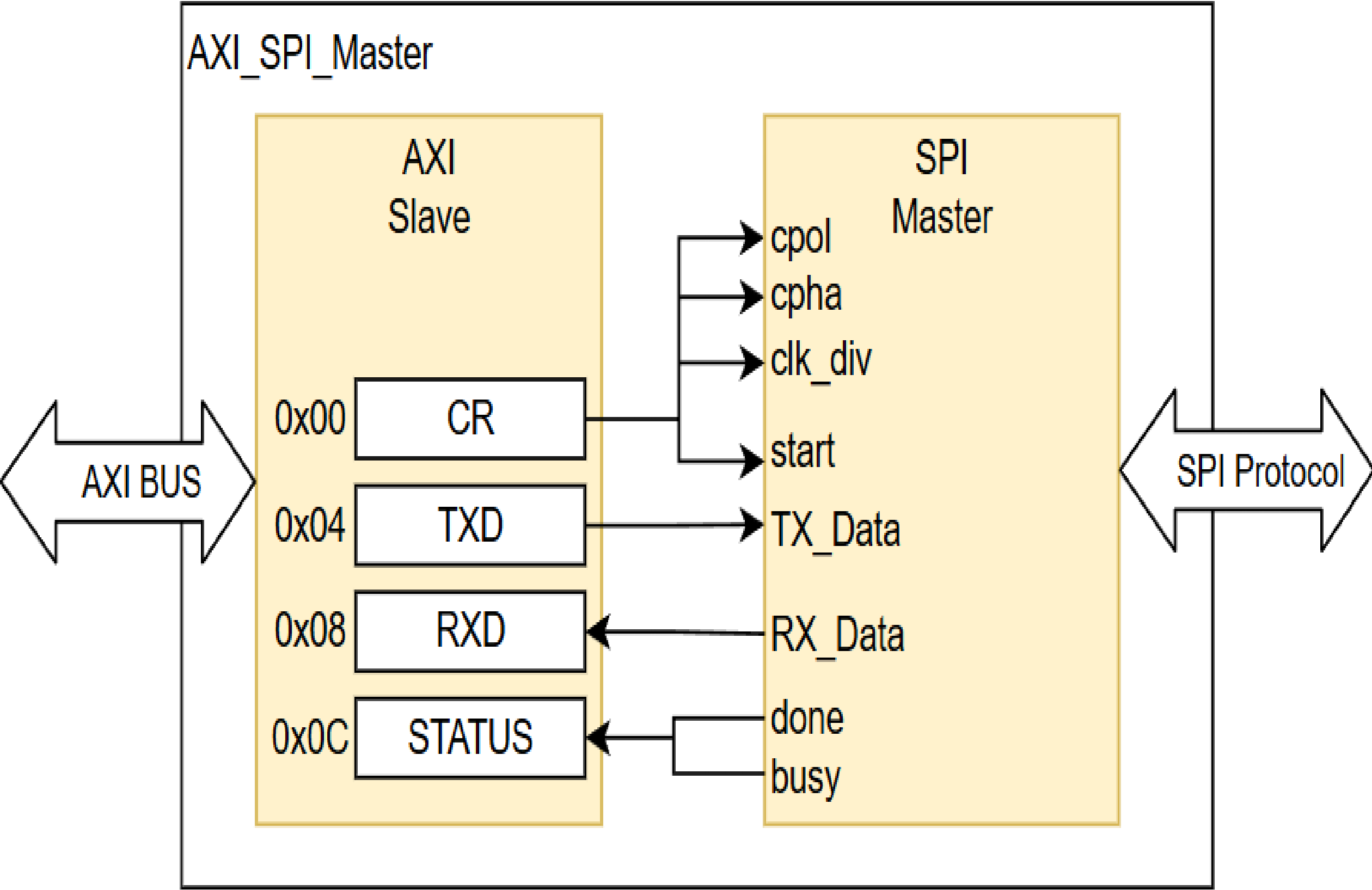


Name	Purpose
SCLK	Serial Clock
MOSI	Data(Master → Slave)
MISO	Data(Master ← Slave)
CS	Slave Select



SPI

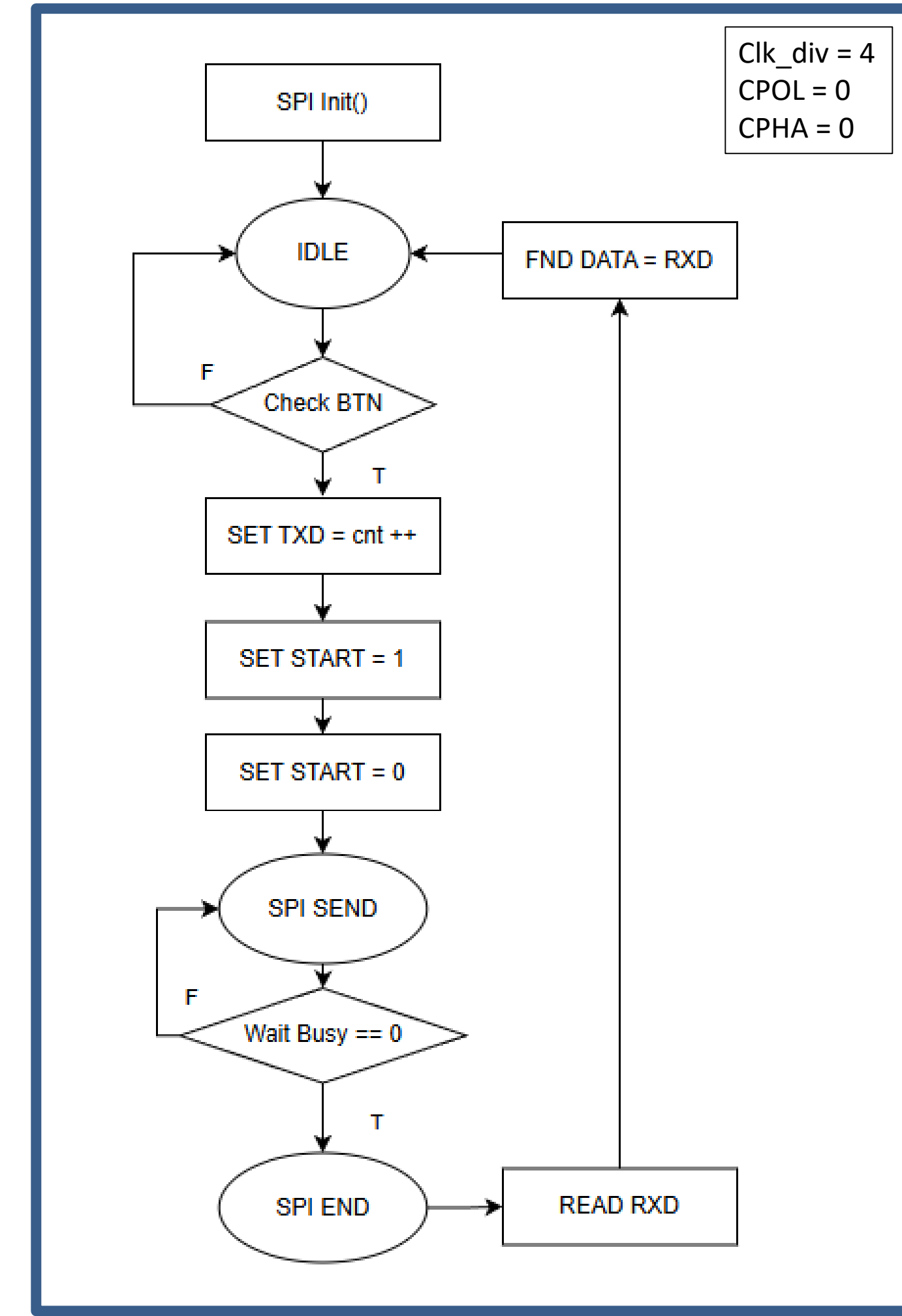
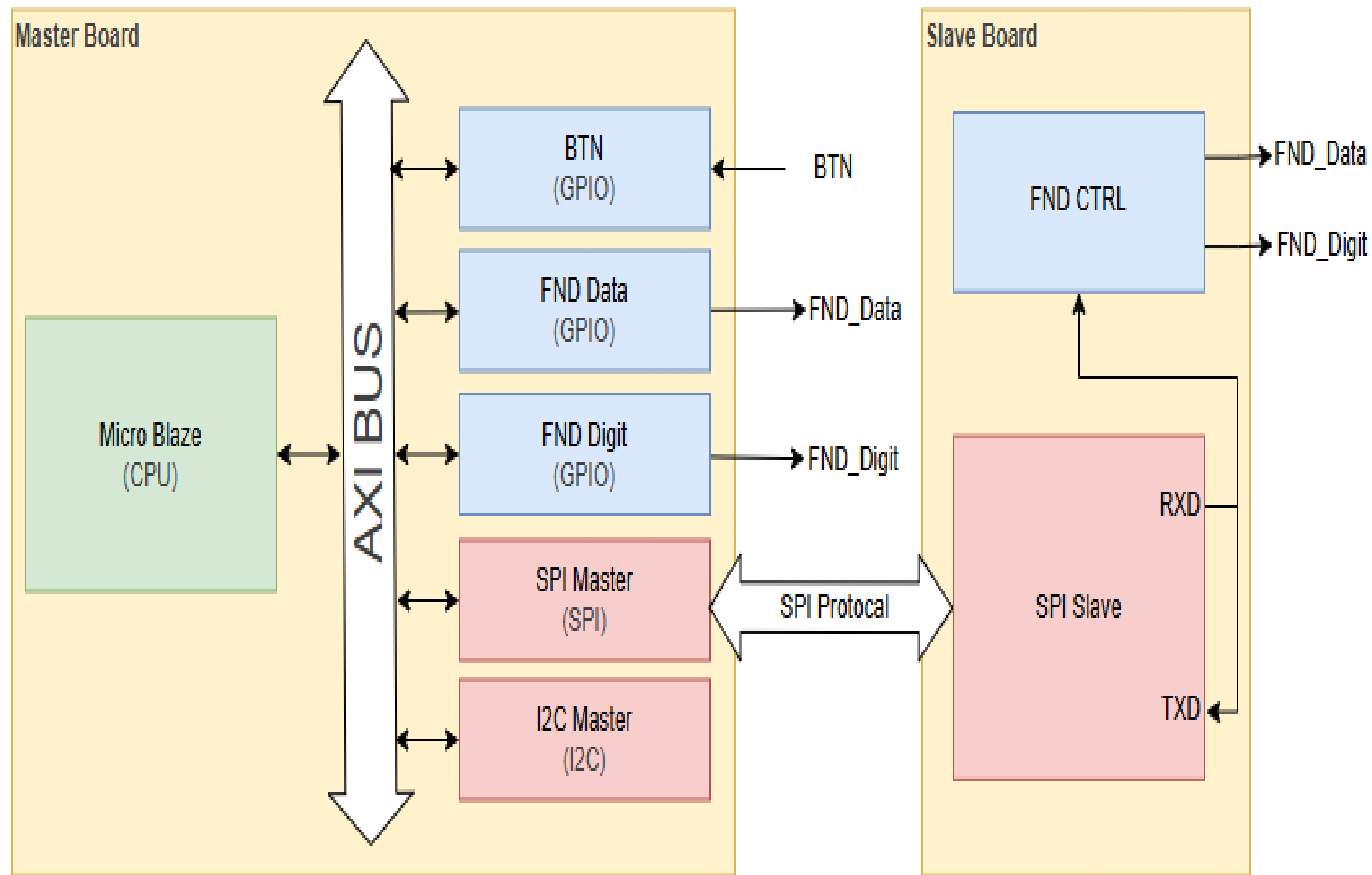
- 모듈 설계



REG	Off-Set	Purpose			
CR	0x00	Start	Clk_div[7:0]	CPHA	CPOL
TXD	0x04	-	TXD[7:0]		
RXD	0x08	-	RXD[7:0]		
STATUS	0x0C	-			Done Busy

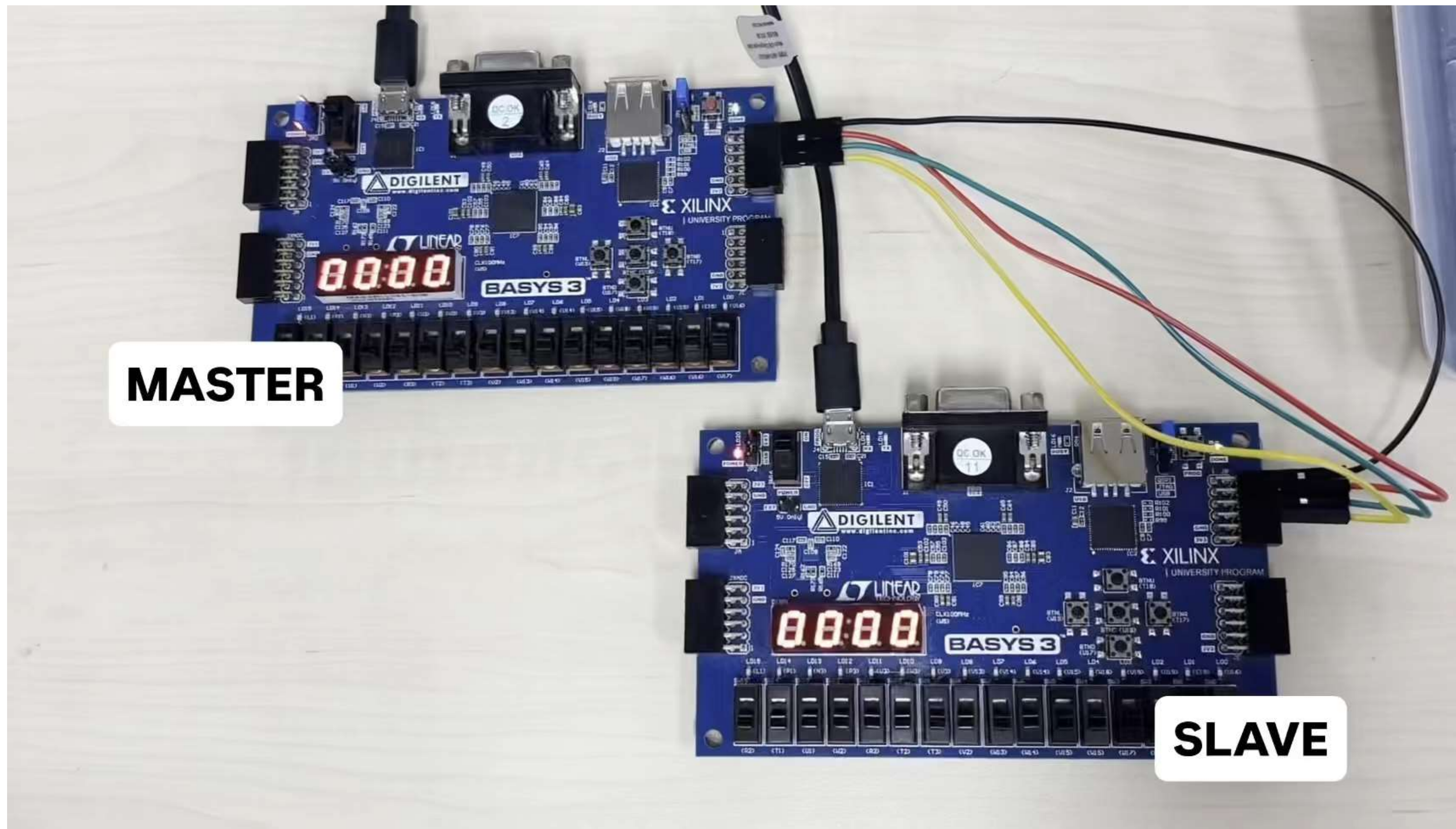
SPI

- Application



SPI

- FPGA 구현



I2C

01 모듈 설계

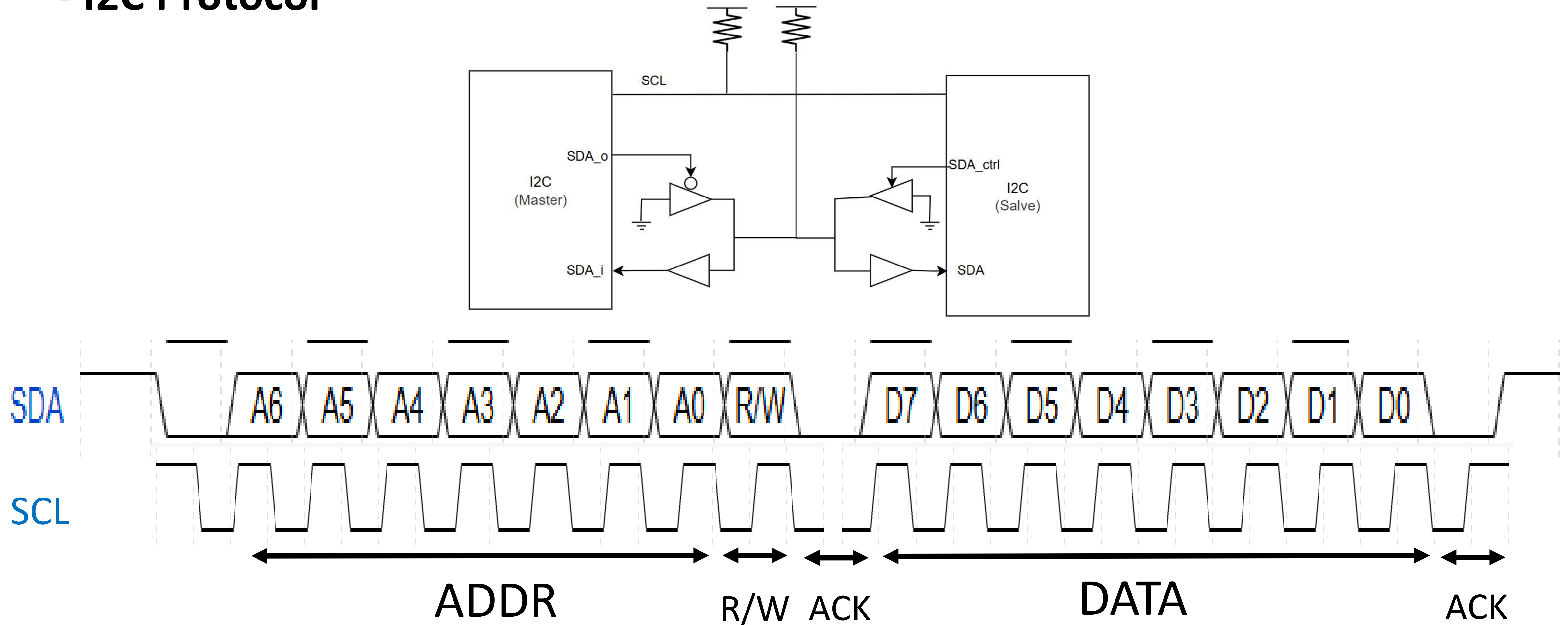
02 Application 설계

03 FPGA 구현



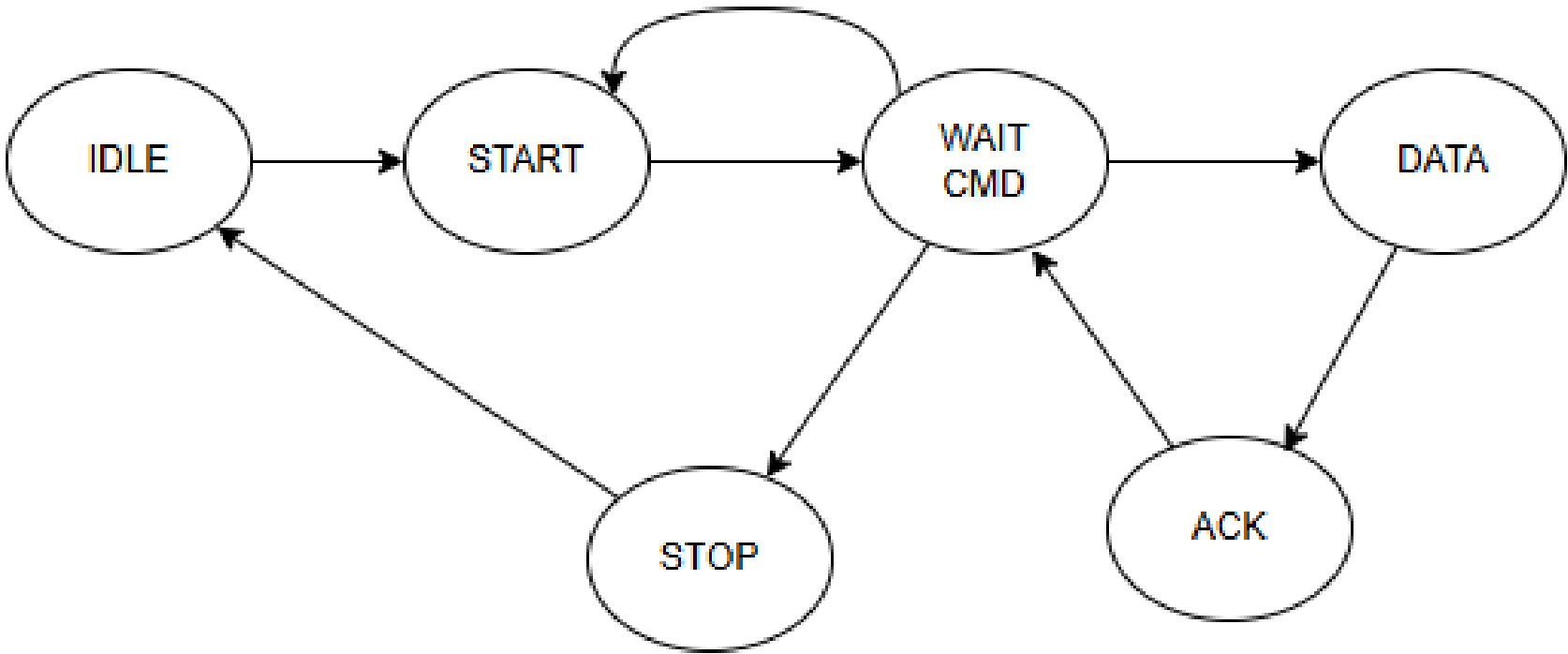
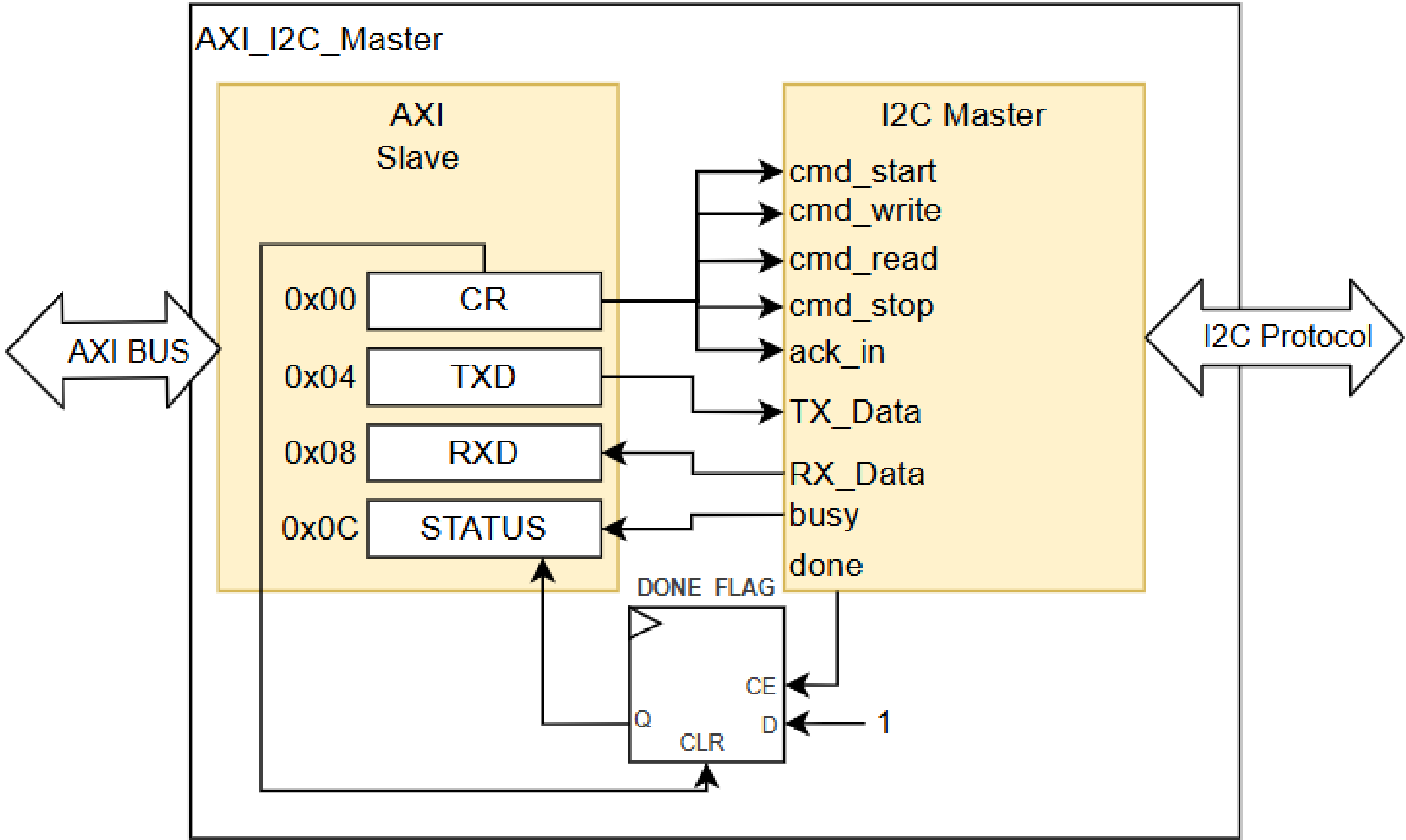
I2C

- I2C Protocol



I2C

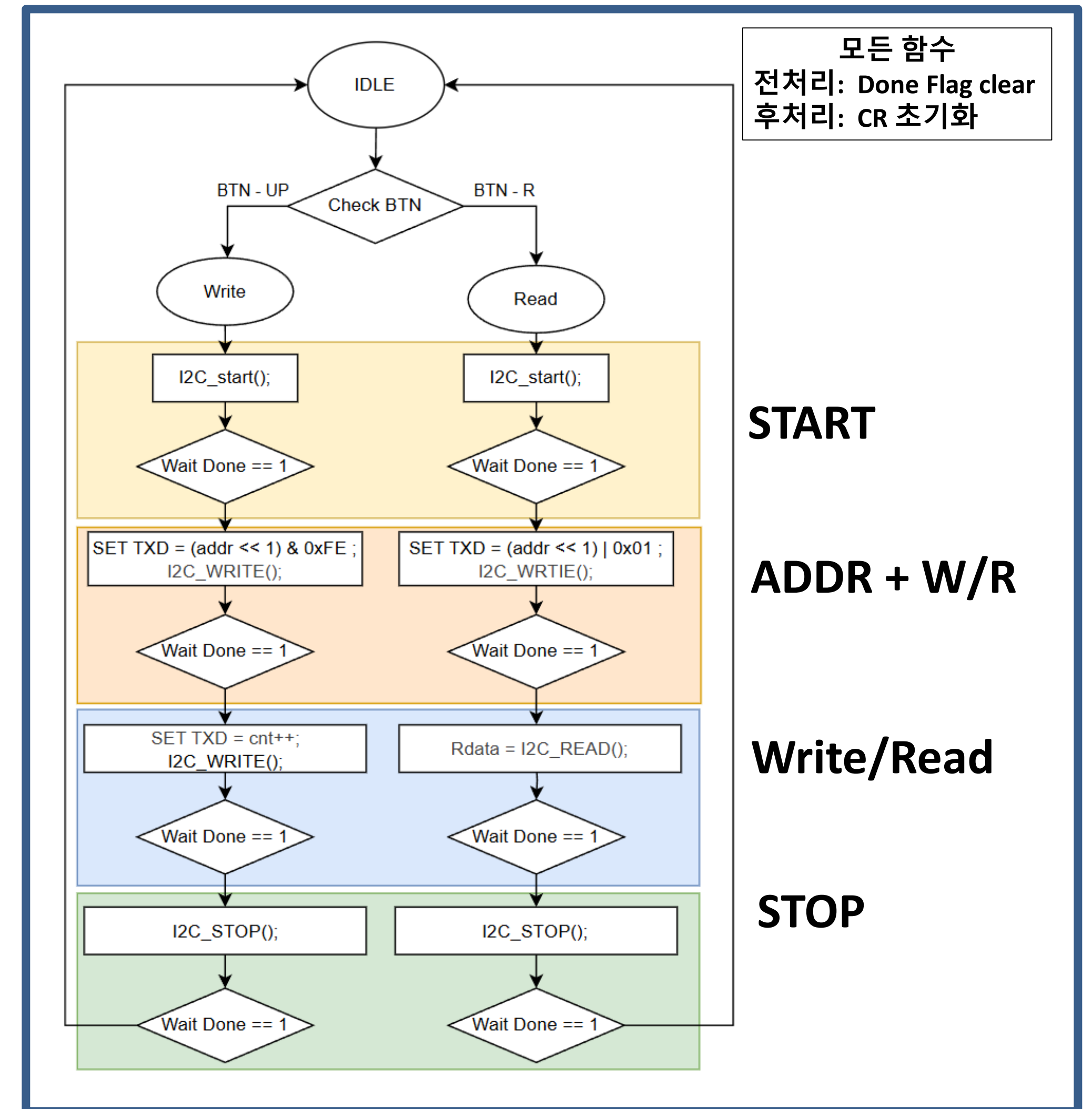
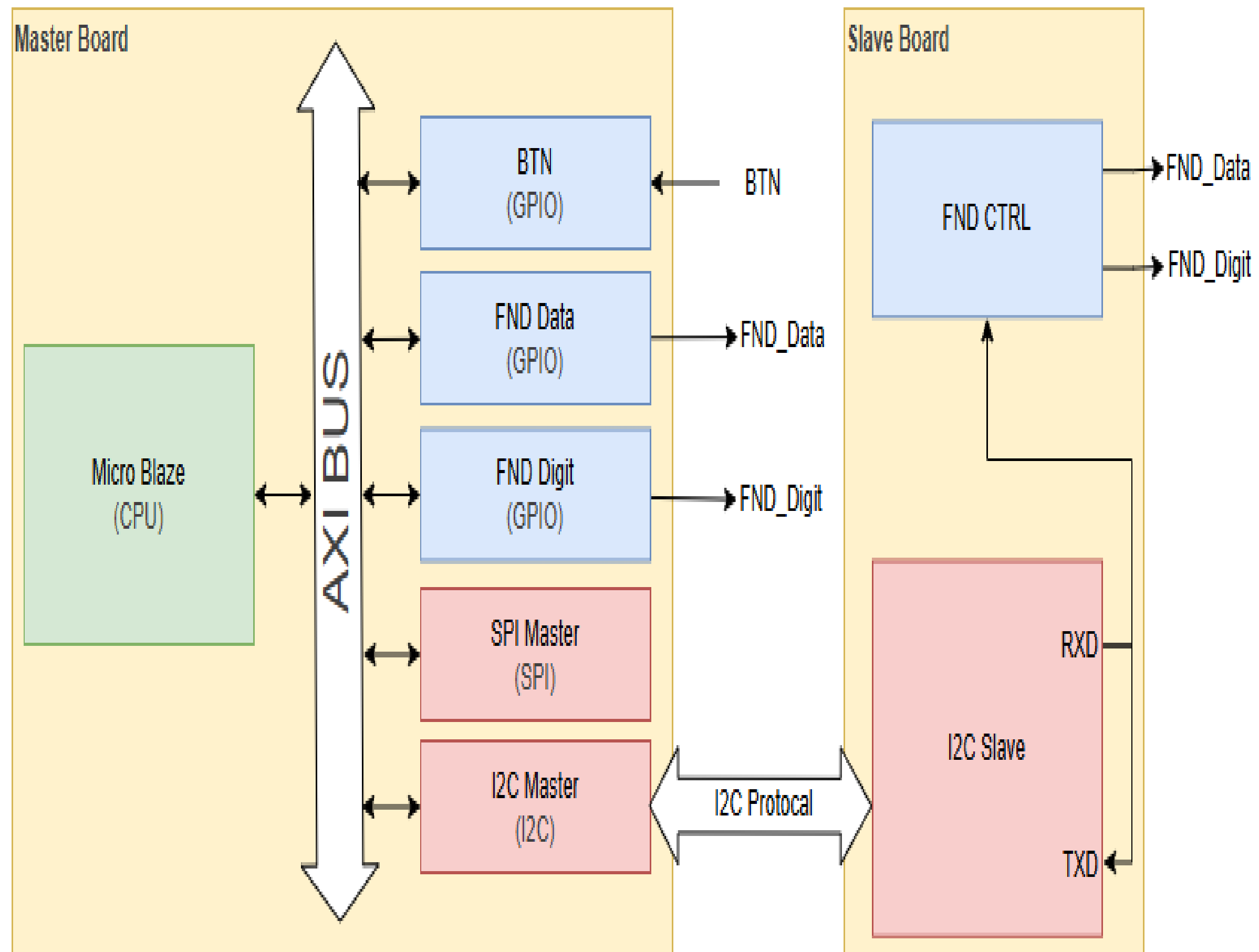
- 모듈 설계



REG	Off-Set	Purpose						
CR	0x00	-	DF CLR	Ack_in	Cmd Stop	Cmd Read	Cmd Write	Cmd Start
TXD	0x04	TXD[7:0]						
RXD	0x08	RXD[7:0]						
STATUS	0x0C	-				Done Flag	Busy	Ack_out

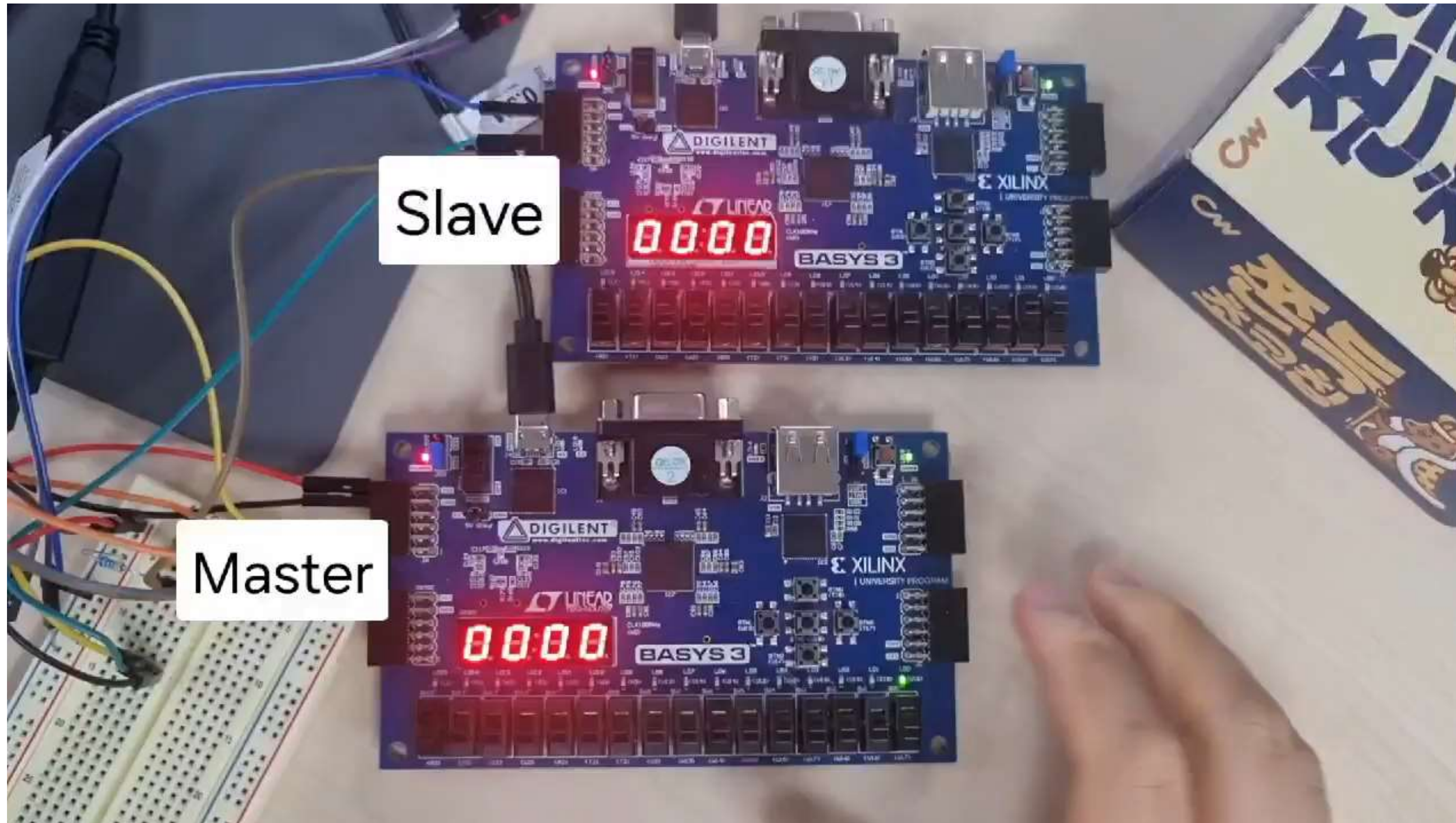
I2C

- Application



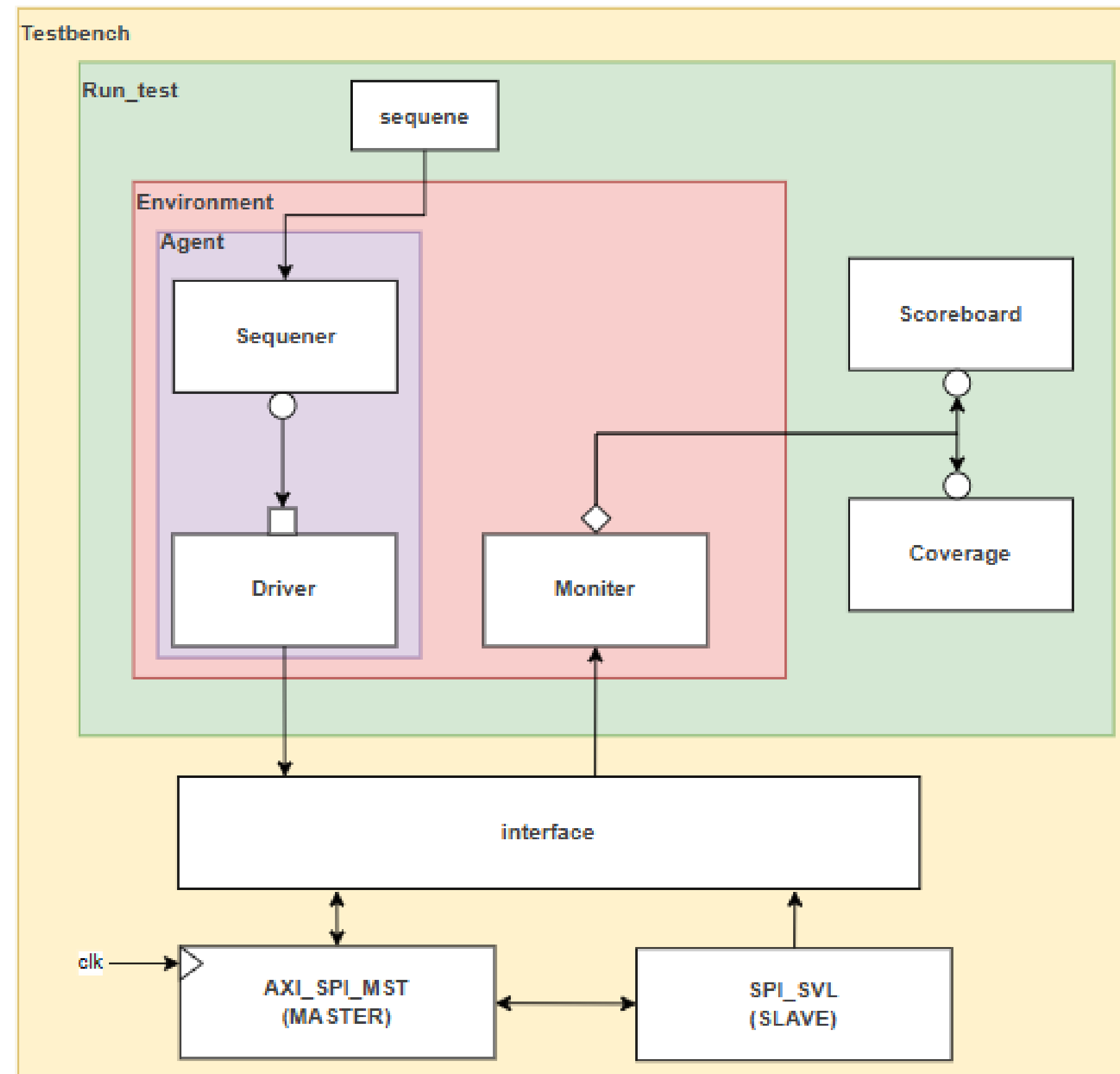
I2C

- FPGA 구현



UVM 검증

- AXI_SPI



검증 할 부분 :

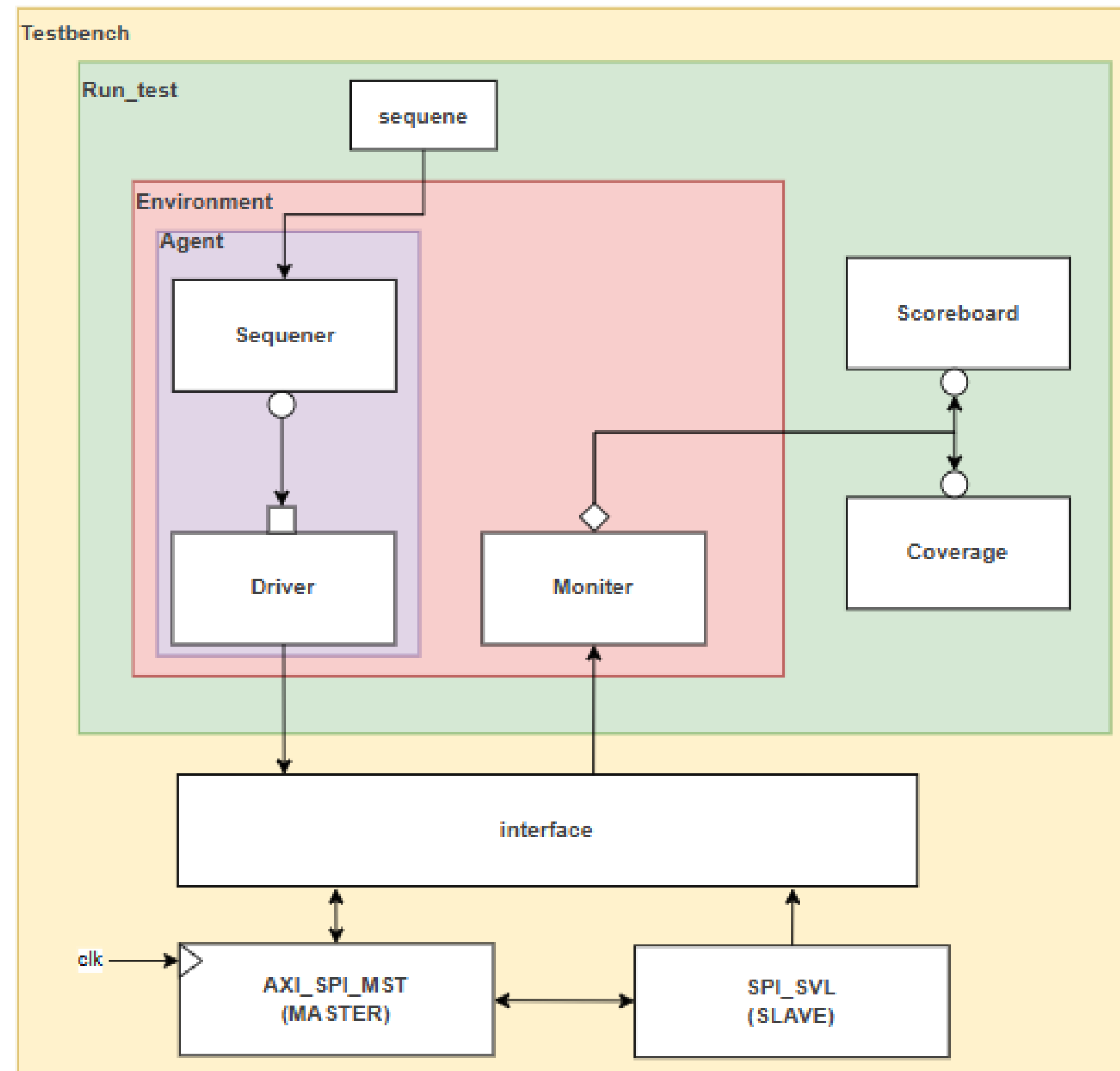
- (AW,W,B,AR,R)채널의 Ready, Valid 신호 Hand Shack 과정 확인
- AXI신호 제어로 SPI Mater의 Tx_data까지 전달되었는지 확인
- AXI신호 제어로 SPI Master START시 통신을 하는지 확인
- SPI 송수신이 잘 이루어지는 확인

Waveform

Scoreboard

UVM 검증

- AXI_SPI : 0~255 data 송수신



Driver :

0. slv_tx_Data = seq값
1. SPI->CR : CLK_div = 4; MODE = 0; 세팅
2. SPI->TXD : seq 값 세팅
3. SPI->CR : SPI START
4. wait(slave_done);
5. SPI->RXD

AXI_Write

AXI_Read

Monitor :

1. AXI_WVALID && AXI_WREADY
Capture axi_wdata, slv_tx_data
2. AXI_RVALID && AXI_RREADY
Capture axi_rdata, slv_rx_data

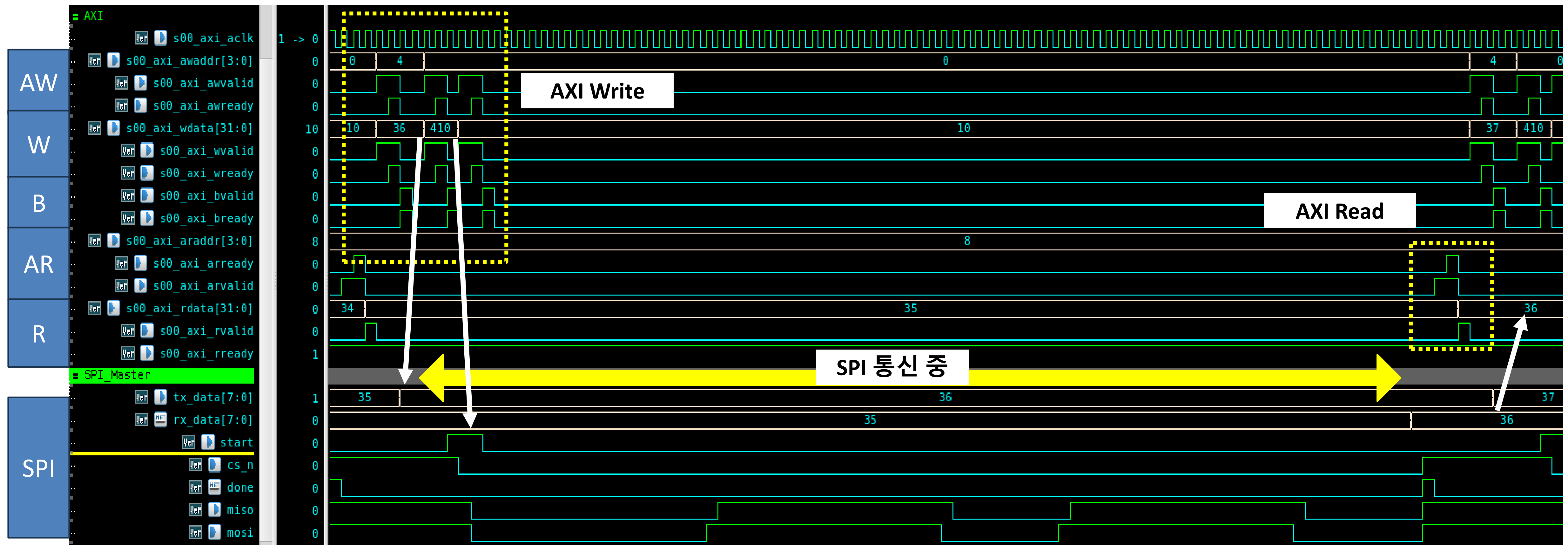
Scoreboard :

1. axi_wdata == slv_rx_data
2. axi_rdata == slv_tx_data

UVM 검증

- AXI_SPI : Waveform (master TX_data = 0x36 인 상황)

순서 : PUSH TDX → START = 1 → START = 0

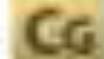
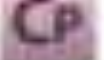


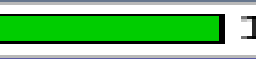
UVM 검증

- AXI_SPI : Report

```
==== Summary Report =====  
Result           : ## PASS ##  
test             : 256  
writes_errors    : 0  
reads_errors     : 0  
  
=====
```

```
UVM INFO /tools/synopsys/vcs/W-202
```

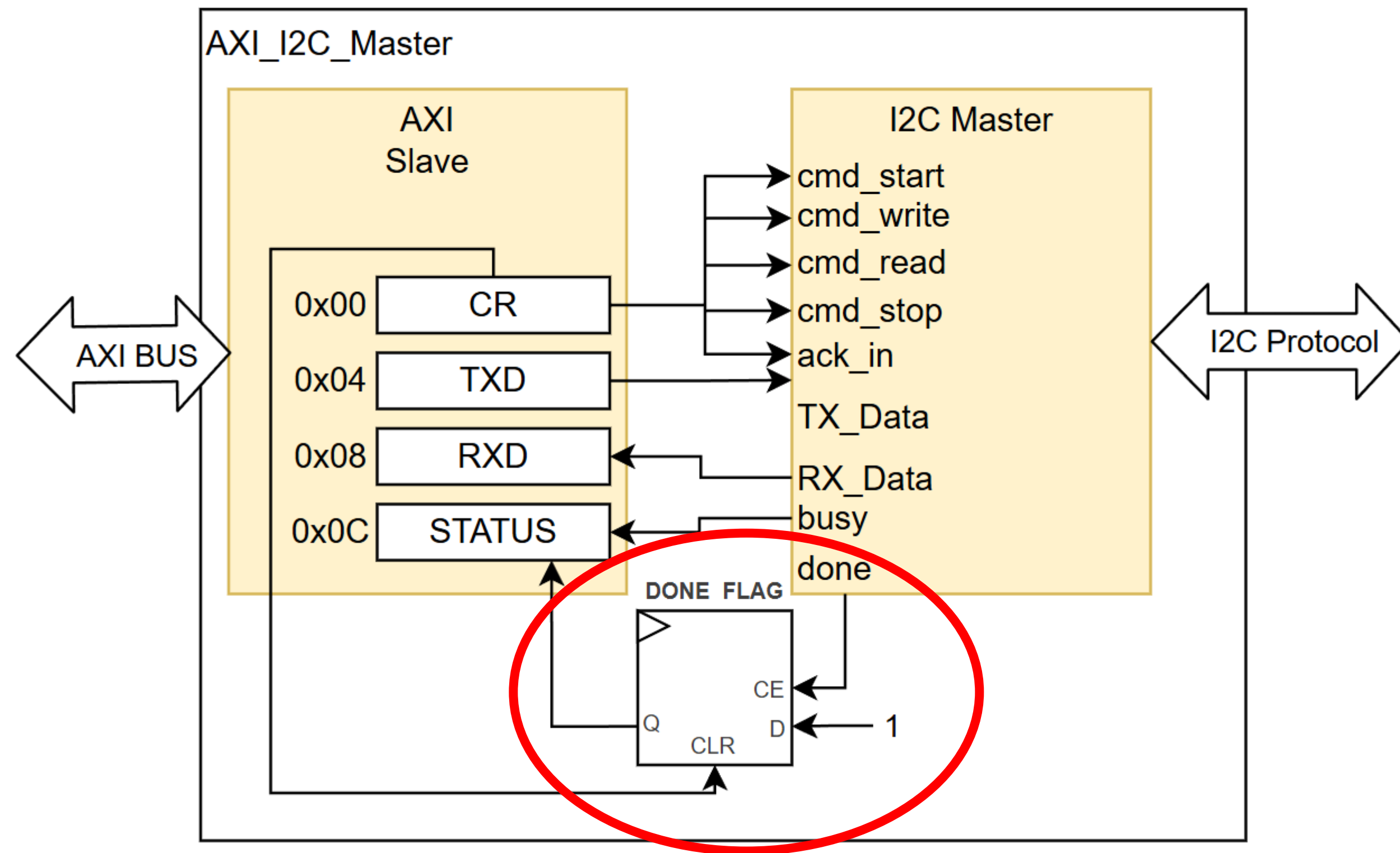
Group	Score	Instances	U+C
 \$unit::AXI_SPI_coverage:...	 100.00%		36
 cp_axi_wdata	 100.00%		18
 cp_slv_tx_data	 100.00%		18

Cover Group Item	Score	Instances	U+C	U	C
 \$unit::AXI_SPI...	 100.00%		36	0	0
 cp_axi_wd...	 100.00%		18	0	0
 cp_slv_tx...	 100.00%		18	0	0

Status	Bin Name	Type	At Least	Size	Hit Count
	FF_val	User	1	1	
	walking_ones_01	User	1	1	
	walking_ones_02	User	1	1	
	walking_ones_04	User	1	1	
	walking_ones_08	User	1	1	
	walking_ones_10	User	1	1	
	walking_ones_20	User	1	1	
	walking_ones_40	User	1	1	
	walking_ones_80	User	1	1	
	walking_zeros_7f	User	1	1	
	walking_zeros_bf	User	1	1	
	walking_zeros_df	User	1	1	
	walking_zeros_ef	User	1	1	
	walking_zeros_f7	User	1	1	
	walking_zeros_fb	User	1	1	
	walking_zeros_fd	User	1	1	
	walking_zeros_fe	User	1	1	
	zero_val	User	1	1	

Trouble shooting

- SW DONE Check



문제:

Done신호에 맞추어 cmd신호를 주려고 시도하였지만 1tick의 done를 SW적으로 확인이 안되는 상황

문제해결:

Done신호를 유지하는 Done Flag Logic를 추가하여 I2C의 done신호를 확인함

느낀 점

과거 하드웨어 설계를 설계하면서 수많은 Trouble Shooting을 겪으면서 문제해결에 대한 감각 키우고 익숙해지고 있었다.
본 프로젝트 Soc설계를 하면서 소프트웨어 설계를 병행하니 새로운 문제를 겪었다.

원인 분석에 많은 시간을 할애 했으며 문제에 대한 해결을 강구했다.
이번에는 많이 도움이 되었던 부분은 소프트웨어의 구조체계 였다고 느꼈다.
분명 구조체계를 몰랐으면 디버깅하는 과정에 난해를 겪어 더 많은 시간을 드렸을 것이다.

이번을 계기로 구조체계의 장점을 이해할 수 있었다. 다른 작업에서도 사용할 수 있는 상황이 오면 적극적으로 사용해 볼 생각이다

